INVENTORY MANAGEMENT SYSTEM



[Inventory management system- GitHub Repository]

Development/ reflection phase
DLBMINPAPCC01_E

REPOSITORY AND REPRODUCIBILITY

Reproducibility:

- Developed on windows using the MSVC developer command prompt
- C source files are compiled using C11
- The C++ entry point is compiled using C++ 17
- Compiled objects files are linked to produce the application executable
- The program is run using *build\grocery_inventory.exe*
- The inventory data file must remain in the expected data/ directory
- The README and codebook help another developer to understand the structure and setup

Repository structure:

src/main.cpp – C++ entry point displaying menus. It receives inputs and directs users to the appropriate operations

src/inventory.c – implements core inventory operations

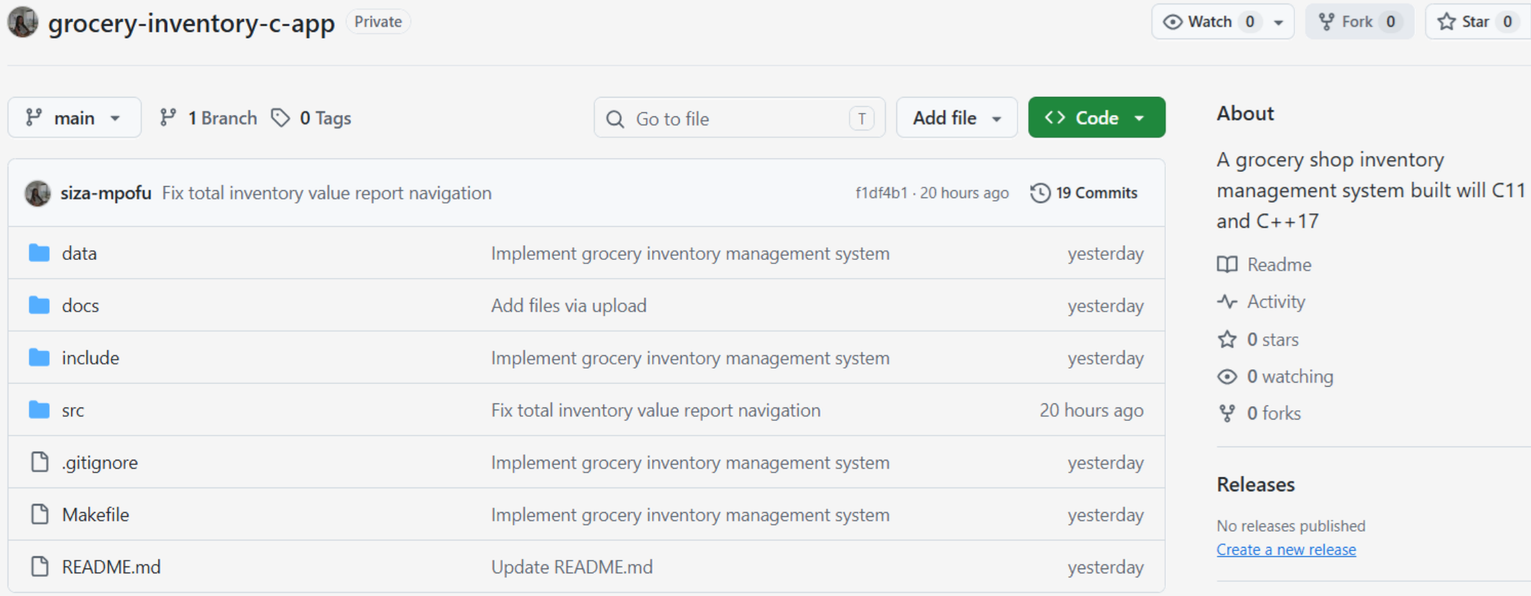
src/persistence.c – handles loading and saving inventory records

src/report.c – implements reporting functions

Include/ - contains header files defining shared data structures and function declarations

data/inventory.txt – stores the inventory records between sessions

docs/development/codebook.md – documents the code files and their responsibilities

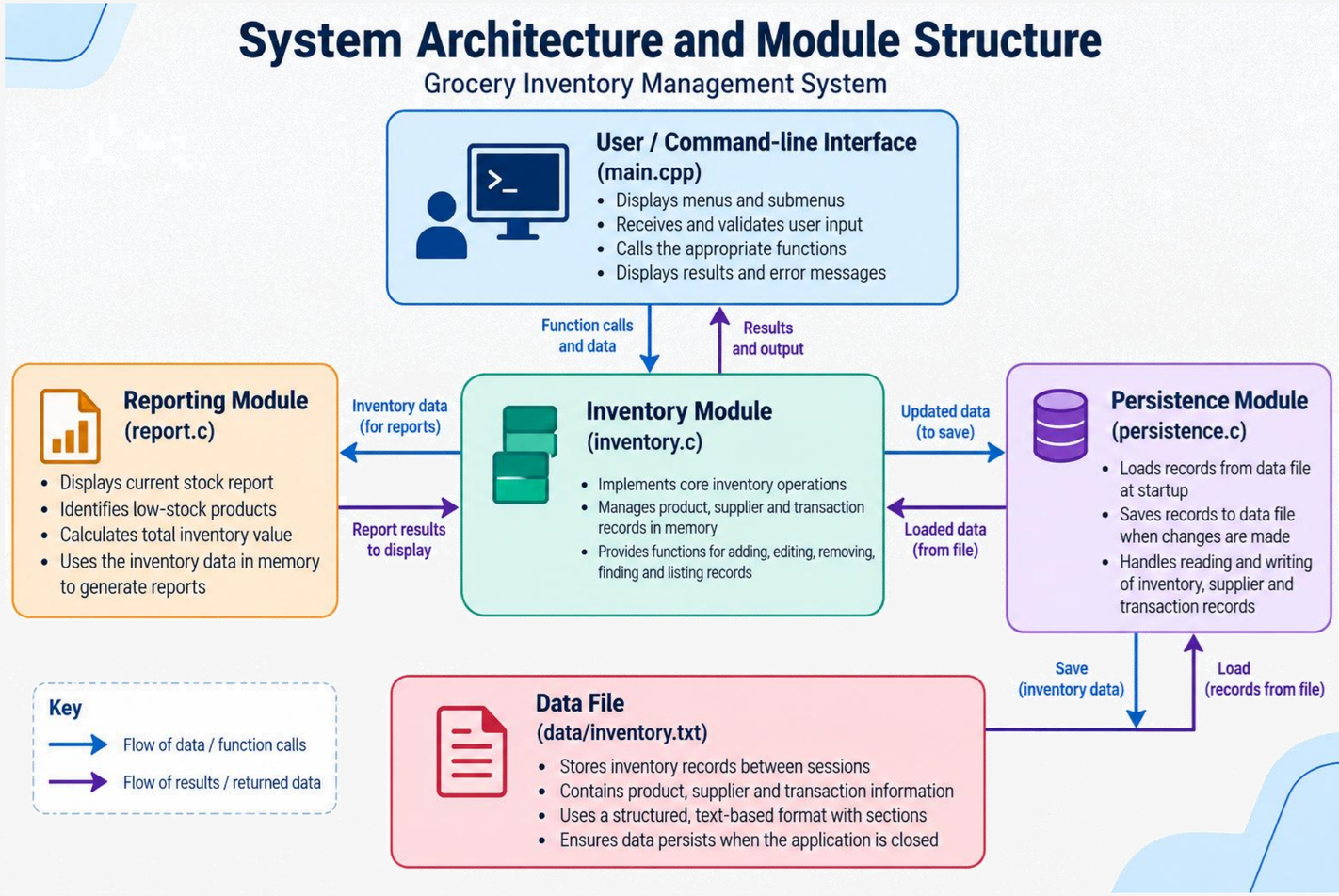


SYSTEM ARCHITECTURE AND MODULE RESPONSIBILITIES

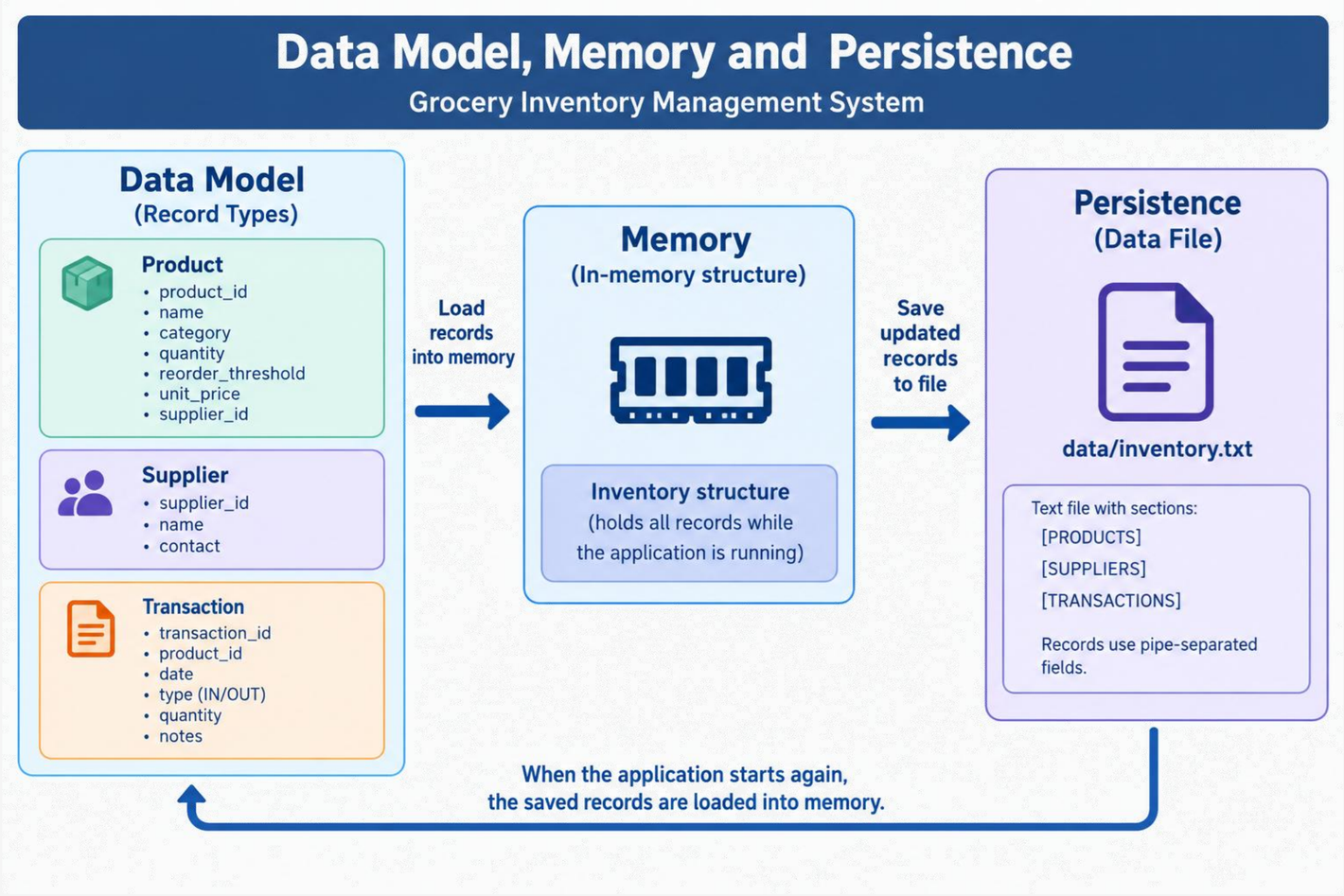
Architecture:

- The application follows a modular architecture
- Each module is responsible for a specific part of the system
- The command line interface communicates with the core inventory functions
- The persistence module connects the applications working data to the text file
- The reporting module produces information from inventory records
- Header files provide shared declarations for communication between modules

- The diagram separates user interaction from core operations, makes individual modules easier to understand and maintain and allows changes to one responsibility without unnecessarily rewriting the whole application



DATA MODEL, MEMORY AND PERSISTENCE



Data model: The system organizes information into three main record types: product, supplier and transaction

Memory: While the application is running, the inventory structure holds the records that the program uses and updates

Persistence: The file data/inventory.txt stores inventory data using pipe-separated fields organized into sections

Data flow: At startup, saved records are loaded into memory. When the inventory is saved, the current records are written to the data file

Continuity: When the application is restarted, the saved records can be loaded again, allowing inventory information to be retained between sessions

IMPLEMENTATION APPROACH AND ALGORITHMS

Hybrid C/C++ implementation:

- C11 is used for inventory operations, persistence and reporting
- C++17 is used for application entry point and menu-driven interface
- The source files are compiled separately using MSVC
- The resulting object files declare structures and functions used across modules
- - Shared header files declare structures and functions used across modules
- C compatible function declarations are needed where C functions are called from C++





Menu-processing algorithm:

- Display the available options
- Read the users selection
- Check whether the input is a valid whole number
- Check whether the selected option is valid for the current menu
- Call the function associated that option
- Display the result or an error message
- Return the relevant menu when appropriate

```
==========================================
 GROCERY SHOP INVENTORY MANAGEMENT SYSTEM
==========================================
1. Products
2. Suppliers
3. Transactions
4. Reports
5. Save inventory
6. Load inventory
0. Exit
Choose an option:
```

IMPLEMENTATION APPROACH AND ALGORITHMS

Low stock algorithm:

- Read the quantity currently available for a product
- Read its reorder threshold
- Compare the two values
- Flag the product when its quantity is below the threshold
- In the example, sugar has a quantity of 7 and a threshold of 8, because 7 is less than 8 sugar appears in the low stock report

```
--- REPORTS ---
1. Current stock
2. Low-stock products
3. Total inventory value
0. Back
Choose: 2

LOW-STOCK PRODUCTS
ID: 1002 | Sugar | Quantity: 7 | Threshold: 8
```

Total inventory value:

- The reporting module calculates a total inventory value from inventory records
- The reporting function returns a value for the interface to display
- Option 3 calls report_total_value(inv)
- The verified output for the tested dataset in the example was 234.01 as shown in the example below

```
--- REPORTS ---
1. Current stock
2. Low-stock products
3. Total inventory value
0. Back
Choose: 3
Total inventory value: 234.01
```

VALIDATION AND EDGE-CASE TESTING

Test	*Input / action*	*Observed result*
Menu validation	Typed text instead of a number	Prompt reported: “Enter a valid whole number.”
Negative quantity	Tried to add a product with quantity −5	Rejected with “Invalid product details.”
Duplicate product	Tried to add Sugar again	Rejected with a duplicate-name message
Low-stock report	Sugar quantity 7; threshold 8	Product correctly appeared in low-stock output
Value report	Selected Reports → option 3	Displayed total inventory value: 234.01

Additional tests to perform include:

- Starting the application when the inventory file is missing
- Testing an empty inventory
- Testing a malformed inventory record
- Verifying how the application reports and handles each condition

The performed tests demonstrate:

- Menu input validation handles non-numeric input
- Product validation rejects the tested negative quantity
- Duplicate handling prevents the tested duplicate product from being added
- The low stock report identifies a product below its reorder threshold
- The menu option produces the intended valuation output
- The saved inventory persists across the tested restart

Lessons learnt:

- Input validation is essential
- Data integrity matters
- Boundary conditions must be tested
- Testing reveals logic errors
- Persistence must be verified

DEVELOPMENT CHALLENGES AND SOLUTIONS

Development challenges and their solutions:

1. Report-menu navigation:

- The reports option needed to call the correct valuation function
- The relevant case in main.cpp was inspected
- Option 3 was updated to call report_total_value(inv)

```
C:\Users\mpofu\Desktop\grocery-inventory-c-cpp>build\grocery_inventory.exe

==========================================
 GROCERY SHOP INVENTORY MANAGEMENT SYSTEM
==========================================
1. Products
2. Suppliers
3. Transactions
4. Reports
5. Save inventory
6. Load inventory
0. Exit
Choose an option: 4

--- REPORTS ---
1. Current stock
2. Low-stock products
3. Total inventory value
0. Back
Choose: 1

CURRENT INVENTORY
------------------------------------------------------------
ID     Product                   Unit       Quantity      Price
------------------------------------------------------------
1001   Sunflower Oil 1L          bottle           49       3.49
1002   Sugar                     5kg               7       9.00
```

2. Invalid user input:

- The application was tested with text entered at a numeric menu prompt
- The program displayed an error message requesting a valid whole number, demonstrating the tested input-validation behavior

```
==========================================
 GROCERY SHOP INVENTORY MANAGEMENT SYSTEM
==========================================
1. Products
2. Suppliers
3. Transactions
4. Reports
5. Save inventory
6. Load inventory
0. Exit
Choose an option: Transactions
Enter a valid whole number.
Choose an option:
```

DEVELOPMENT CHALLENGES AND SOLUTIONS

3. Product validation:

- A negative product quantity was tested
- The application rejected the invalid product details
- A duplicate product was also tested and rejected

```
Product name: Rice

1. Oil
2. Rice
3. Sugar
4. Flour
5. Canned
6. Drinks
7. Other
Choose category: 2
Unit: 2kg
Opening quantity: -5
Low-stock threshold: 7
Unit price: 5
Invalid product details.
```

```
Product name: Sugar

1. Oil
2. Rice
3. Sugar
4. Flour
5. Canned
6. Drinks
7. Other
Choose category: 3
Unit: 5kg
Opening quantity: 7
Low-stock threshold: 8
Unit price: 9
Could not add product. Check for duplicate names.
```

4. Persistence verification:

- The inventory was saved before the application exited
- The application was reopened
- The current stock report was checked
- Sunflower oil 1L remained at 49 and sugar remained at 7

```
==========================================
 GROCERY SHOP INVENTORY MANAGEMENT SYSTEM
==========================================
1. Products
2. Suppliers
3. Transactions
4. Reports
5. Save inventory
6. Load inventory
0. Exit
Choose an option: 0
Inventory saved. Goodbye!
```

```
C:\Users\mpofu\Desktop\grocery-inventory-c-cpp>build\grocery_inventory.exe

==========================================
 GROCERY SHOP INVENTORY MANAGEMENT SYSTEM
==========================================
1. Products
2. Suppliers
3. Transactions
4. Reports
5. Save inventory
6. Load inventory
0. Exit
Choose an option: 4

--- REPORTS ---
1. Current stock
2. Low-stock products
3. Total inventory value
0. Back
Choose: 1

CURRENT INVENTORY
------------------------------------------------------------
ID     Product                 Unit       Quantity      Price
------------------------------------------------------------
1001   Sunflower Oil 1L        bottle           49       3.49
1002   Sugar                   5kg               7       9.00
```

SUMMARY, LESSONS LEARNT AND FUTURE IMPROVEMENTS

Implemented and demonstrated:

- Modular grocery inventory application using C11 and C++17
- Product, supplier and stock management workflows
- Current stock report
- Low stock report
- Total inventory value report
- Validation of tested invalid inputs and duplicate product
- Saving and reloading inventory records across application restarts

Lessons learnt:

- Code changes should be followed by rebuilding and testing
- Compilation alone does not prove that a feature works correctly
- Actual program output provides evidence that a correction works
- Both valid and invalid input conditions should be tested

Future improvements:

- Expand grocery product dataset
- Add automated tests for missing, empty and malformed files
- Improve error handling and recovery
- Improve data validation and reporting coverage
- Consider a database if the application needs multi-user access
- Enhanced searching and sorting

Conclusion:

- The application demonstrates core inventory workflows, reporting, input validation and persistence
- Further testing and stronger error handling would improve reliability
- The project can be extended with more comprehensive grocery shop workflows